Decodable, Not Transferable: Latent Actions Across Worlds

Alessio Colombo
alessio.colombo@student.uva.nl

May Lee*
may.lee@student.uva.nl

Elyanne Oey*
elyanne.oey@student.uva.nl

Isabelle A. Kampono*
izzi.kampono@student.uva.nl

Stipe Frkovic
stipe.frkovic@student.uva.nl

Dheeraj Varghese
d.varghese@uva.nl

Cees G. M. Snoek
C.G.M.Snoek@uva.nl

University of Amsterdam

Abstract

Latent action models aim to make unlabeled video useful for learning controllable world models by inferring latent actions from visual transitions. For these latents to serve as reusable action representations, however, their semantics must remain consistent across changes in visual context and environment. We ask what action-related structure can be identified in pretrained latent action model representations, and whether this structure can support action prediction in known and unknown environments. We answer it as three nested questions: (i) how is action information *encoded* in the latents, (ii) is it *decodable* into actions inside a seen environment, and (iii) does the same decoder *transfer* to an environment never seen during probe fitting? We evaluate two recent latent action models, AdaWorld and Olaf-World, on a 193-game Stable Retro corpus and a subset of Open Pixels2Play. We find that action labels can be predicted from the latents, but the signal is weakly linearly separable and entangled with game-related variation. There is a substantial drop in action prediction performance on unseen environments compared to known environments, mostly for nonlinear probes. Moreover, a generic video tokenizer carries the strongest *within-game* action signal of any representation we test, suggesting that such decodability may reflect generic visual dynamics rather than an action-specific inductive bias of latent action models.

1 Introduction

World models learn predictive representations of environment dynamics, enabling agents to plan and act by simulating future states internally [Ha and Schmidhuber, 2018, Hafner et al., 2019] . In interactive settings such as game playing, robotic control, and embodied navigation, these models must additionally account for the actions taken by the agent: given a current state and an action, the

*Equal contribution.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: PTA: From Pretrained Representations to Acting Agents.

model should predict the resulting next state. Most existing world models assume access to action labels provided by the environment [Micheli et al., 2022, Hafner et al., 2023]. This assumption limits their ability to exploit the vast amount of video data for which no corresponding action annotations are available.

Latent action models (LAMs) address this limitation by learning an action space directly from unlabeled video. Rather than conditioning dynamics on externally provided actions, they infer a compact latent variable from consecutive observations and use it to model the transition between them. Genie [Bruce et al., 2024] demonstrated this approach for interactive world modeling, and subsequent methods including LAPA [Ye et al., 2024], AdaWorld [Gao et al., 2025], and Olaf-World [Jiang et al., 2026] have further developed the architecture and training procedure. Beyond providing a conditioning signal for prediction, these learned latents are often interpreted as capturing action-like structure: reusable primitives corresponding to the underlying changes induced by an agent. *But what do these latent actions actually represent?* Ideally, a latent action should represent the action responsible for a transition consistently. In particular, if the representation recovers atomic actions, then the identity of an action should remain recognizable across different states and environments. For example, a representation of "move left" should not acquire an entirely different meaning simply because the character, background, camera configuration, or game has changed.

Recent work complicates the interpretation of latent actions as action primitives, showing that learned latents can encode properties of visual change and camera motion [Garrido et al., 2026, Zhang et al., 2025, Wang et al., 2026a]. What remains unclear is whether the underlying action is itself represented consistently across different states and environments. We study this question in video games, where the executed action is known and can be observed repeatedly across highly varied states and environments. This provides a controlled setting for testing whether representations learned from video recover the atomic actions that generated the transitions. We evaluate AdaWorld [Gao et al., 2025] and Olaf-World [Jiang et al., 2026] on 3D game environments from a P2P subset [Yue et al., 2026] and on a custom dataset spanning 193 2D retro games collected with Stable Retro [Poliquin, 2026]. We additionally evaluate VideoFlexTok [Atanov et al., 2026] as an alternative approach.

We evaluate the atomic-action hypothesis through three questions. **Q1:** How is action information encoded in the latents? **Q2:** Is it decodable into actions inside a known environment? **Q3:** Does the same encoder transfer to new environments? Figure 1 previews our answers to all three. Cross-environment consistency is particularly important if latent actions are to serve as reusable action primitives, but we do not find this property. To our knowledge, we provide the first systematic evaluation of whether learned latent-action representations preserve a consistent latent action mapping across a diverse collection of game environments.

2 Evaluation Setup

2.1 Models

We evaluate three models. AdaWorld and Olaf-World both learn continuous latent representations of visual transitions, but differ in how the latent action space is constrained. VideoFlexTok is a general-purpose video tokenizer that we repurpose as a latent action model.

AdaWorld [Gao et al., 2025] learns latent actions with a Transformer autoencoder comprising a spatiotemporal encoder and a spatial decoder. Given two frames, the encoder compresses their visual change into a 32-dimensional continuous latent, and the decoder predicts the second frame conditioned on the first frame and this latent. The model is trained with a β-VAE objective, imposing an information bottleneck on the latent action. From the released checkpoint, we extract $z_t \in \mathbb{R}^{32}$ for each frame pair at 256×256 resolution.

Olaf-World [Jiang et al., 2026] introduces SeqΔ-REPA, a sequence-level alignment objective that aligns sequence-averaged latent actions with temporal feature differences from a frozen V-JEPA 2 encoder, promoting consistent action semantics across visual contexts. Its latent action encoder is likewise a spatiotemporal Transformer, with causal temporal attention. We use the released `align` variant, which adds SeqΔ-REPA to the latent-action reconstruction objective. The checkpoint is trained at 272×480; we extract P2P latents at this resolution and Retro latents at 256×256.

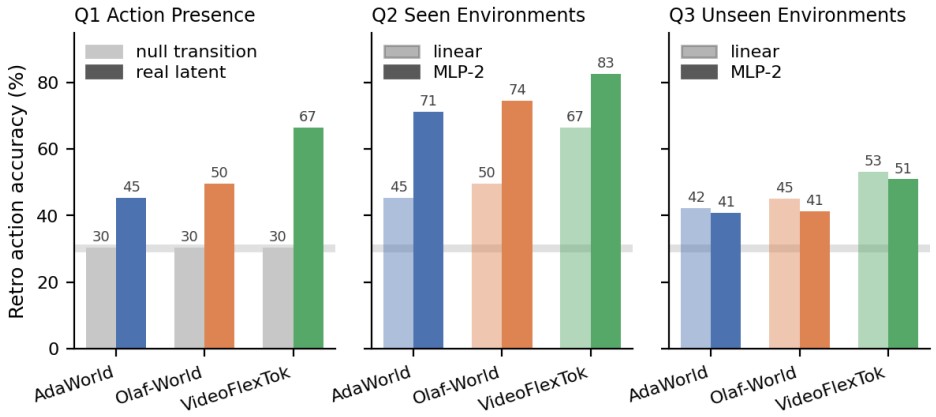


Figure 1: **Action signal exists, but not as a primitive.** We plot action accuracy for the three encoders on the Retro dataset. **Q1**: Action information is present: a linear probe predicts actions better than the majority baseline for every encoder. **Q2**: Actions are decodable inside a known environment, and probe depth improves performance. **Q3**: The same weights as in **Q2**, scored on held-out games, lose everything depth bought. Gray band: majority baseline of the two test sets (29.5–31.0%). Action representation does not transfer between games.

VideoFlexTok [Atanov et al., 2026] resamples 3D VAE latent frames into a coarse-to-fine sequence of up to 256 tokens per latent frame. Each latent frame corresponds to 4 input frames under the VidTok VAE's temporal compression. To repurpose VideoFlexTok as a latent action model, we take the latent frame aligned with the 4 action frames and retain either the first 32 or all 256 tokens, yielding flattened representations in $\mathbb{R}^{32\times6}$ and $\mathbb{R}^{256\times6}$, respectively, at 256×256. We use the continuous pre-quantization FSQ vectors rather than the discrete FSQ codes, keeping the representation continuous as in AdaWorld and Olaf-World.

More details on the model architectures and checkpoint sources are collected in supplemental Appendix A.1.

2.2 Datasets

Retro Dataset. We construct a dataset from 193 retro platformer games using Stable Retro [Poliquin, 2026], a maintained fork of OpenAI's Gym Retro. For each game, we collect 20 rollouts of 4,000 steps using a scripted controller adapted from GenieRedux [Savov et al., 2025]. In each 20-frame cycle, the controller idles for 16 frames and then holds one randomly sampled action for 4 frames, making the resulting visual change easier to attribute to that action (see Limitations; Figure 4; construction details in Appendix A.2). The two frames handed to an encoder are *non-consecutive*, and the gap between frames depends on the class (Appendix A.3). We retain four controller labels: `right`, `left`, `crouch`, and `jump`, yielding a four-class classification task.

P2P Dataset. We further report on the Open Pixels2Play (P2P) dataset [Yue et al., 2026], which consists of gameplay video from 3D platformer games with action annotations. Each frame pair carries a subset of 10 keyboard labels; because several keys can be held simultaneously, action prediction is multi-label. From the full P2P release, we use the 291 sessions captured at 640×480, comprising 999,720 frame pairs across 15 games, split 80/20 (Appendix A.4). We evaluate AdaWorld and Olaf-World on both P2P and Retro. VideoFlexTok is evaluated only on Retro, as its input format requires multi-frame clips rather than two-frame transitions.

2.3 Experiments

We probe for each question in turn: **Q1:** How is action information encoded in the latents? **Q2:** Is it decodable into actions inside a known environment? **Q3:** Does that decoder transfer to an environment never seen during probe fitting?

Q1: We perform two analyses: a) strength of action encoding and b) structure of action encodings.

a) We train linear and MLP-d[2] probes to predict action labels on Retro and keys on P2P from latents pooled over all games and environments, and compare them against probes trained on *null* pairs, in which both frames are identical so the input carries no transition. On P2P (multi-label, 10 keys) we report exact-match accuracy, Hamming loss and per-key F1.

b) We measure action *entanglement* between action classes and with games. An axis-feature Fisher discriminant ratio $F = \sigma^2_{\text{between}}/\sigma^2_{\text{within}}$ over action classes ($F \gg 1$ marks a dimension that separates actions), and inter- versus intra-action centroid distances, each against the null pairs. For the latter, over the whole Retro corpus, we take the average pairwise L2 distance between latents that share an action but differ in game and between latents that share a game but differ in action.

Finally, we test whether *independently* recoverable action information is obscured by *feature superposition*: whether action information is encoded in a distributed and non-feature-aligned form in the original latent basis. We attempt to recover linear action-relevant directions by training sparse autoencoders (SAEs) [Cunningham et al., 2023] with a top-k sparsity loss (Appendix B.1) to learn a sparse reparameterization of the latents. We repeat probing to monitor for improvements in linear separability. To test feature-level alignment directly, we score each sparse feature's Pearson correlation and Fisher discriminant against action and game labels, comparing against corresponding scores on the original encodings. In all SAE experiments, game-label analysis provides a positive control.

Q2: We train two kinds of action probe on the same task. A single *pooled* probe is trained on latents from all games or environments together, and evaluated on a random 80/20 within-environment split (split details in Appendix A.5): since every game contributes to training, this probe can partly succeed by recognizing *which game* it sees, rather than reading the action alone. We separately fit *per-environment* probes, one per Retro game (or P2P environment), each trained and tested only on that environment's own samples, to see how much accuracy varies from game to game (Appendix C.1).

Q3: To test whether a probe generalizes to games it has never seen, we hold out 39 of the 193 Retro games entirely, so the probe never sees a single sample from them during training. We fit one probe on 80% of the samples from the remaining 154 games and score it two ways: *within*, on the held-out 20% of those same 154 games, and *cross*, on the 39 unseen games. Because Retro's four action classes are unbalanced, we report accuracy against each test set's own majority-class baseline rather than the 25% chance level (30.6–31.1% within, 29.4–29.5% cross; complete majority baselines in Appendix Table 8).

3 Q1: How is action information encoded in the latents?

Action labels can be predicted from the transition latents. Table 1 compares probes trained on our latents against probes trained on *null* pairs, in which both frames are identical so the input carries no transition. We observe that action accuracy falls to the majority baseline at every probe depth and for every encoder, while game accuracy is undamaged (Table 2, last row). Only the action accuracy disappears, so null-transition latents are still informative. Every encoder's real latent can predict actions well above baseline. This control shows that the label is predicted from the visual transition. Also, game identity is decodable at up to 99.2% accuracy and the latents support strong visual content prediction (Table 2, Appendix Table 21), so scene information is far easier to recover than the action signal.

Latents describe the scene, and the action is a small, overlapping residual on top of it. Again from Table 1, we see that a handful of directions carry almost all of either LAM's variance on Retro: 4 of 32 dimensions for AdaWorld, 16 of 32 for Olaf-World. This is not just a dimensionality artifact: VideoFlexTok needs four-fifths of its far larger 1,536-d space to reach the same 99% (Appendix D.1). Repeating the measurement on null pairs shows why: almost all of that structure remains even with no action taken, and a real transition adds only two or three directions on top. So these are state descriptors with a thin transition-dependent residual, not action codes that merely retain some context.

The four action classes are also not far apart: their centroids sit closer together (0.22–0.42 AdaWorld, 0.25–0.43 Olaf-World; Table 1) than two samples that differ only in the action taken (0.80 and 1.07 respectively; Appendix Table 20). This means the class means differ little compared to the sample-to-sample variation, which fits the low linear accuracy. P2P shows the same pattern.

[2]Throughout, MLP-d denotes a probe with d hidden layers, so MLP-0 is the linear probe.

Table 1: **Action signal is above the null-transition floor, but no single latent coordinate captures it.** The *null transition* action latents are extracted from input containing no transition, and they sit at the majority baseline at every probe depth. *Fisher* is the best single coordinate's discriminant ratio over the four classes; $F \ll 1$ everywhere means that the signal is not axis-aligned. *Action centroid distances* are the smallest and the largest of the six pairwise L2 distances between the four class means; they are comparable down a column but not across encoders of different dimensionality, which is why VideoFlexTok is left out of that row. The last two rows count the principal components carrying 99% of the latent's variance.

	AdaWorld	Olaf-World	VideoFlexTok
Null transition, linear	30.3	30.4	30.4
Null transition, MLP-1	30.7	30.4	29.9
Null transition, MLP-2	30.6	30.6	29.3
Real latent, linear	45.1	48.7	**65.6**
Fisher	0.097	0.132	0.401
Action centroid distances, min–max	0.22–0.42	0.25–0.43	—
PCA components for 99% of variance	4 / 32	16 / 32	1218 / 1536
Same for the null transition	1 / 32	14 / 32	1180 / 1536

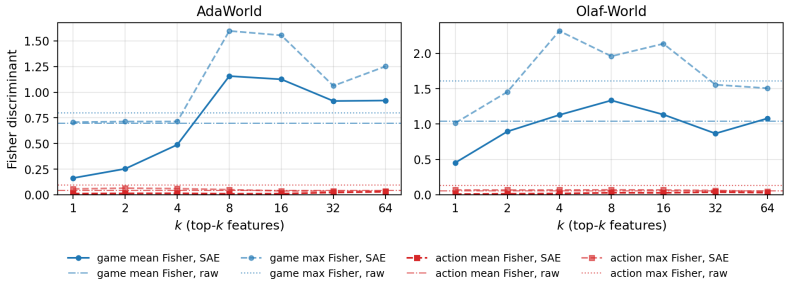


Figure 2: **SAEs recover per-feature directions for games but not for actions.** Plots of Fisher discriminant of the highest-scoring feature and the mean over the top ten, against game (blue) and action (red) labels along the top-k sweep for AdaWorld (left) and Olaf-World (right). A high Fisher score indicates that a label is separable across feature coordinates. Game Fisher climbs with k and passes its raw reference (dotted blue lines) on both encoders, showing recovered directions, while action Fisher scores remain near zero at every k. Although occasionally exceeding the raw Fisher reference (dotted red/orange lines)—most clearly for Olaf-World—the gains are modest, peaking at 0.071. The VideoFlexTok scores show a similar pattern (Appendix Table 11).

It is difficult to linearly separate actions in latent space. The Fisher row of Table 1 says that no single coordinate of the raw latent separates the four action classes, which leaves two readings: the latent may lack a strong linearly recoverable action basis, or it may hold that structure in a distributed, potentially superposed form across feature dimensions. The SAEs tested for the second possibility, but a sparse reparameterization was unable to concentrate action information into a few, individually action-selective features. Linear action accuracy on the SAE codes (Appendix B.3) stays flat as k increases, never exceeding the raw latent by more than a fraction of a point. At the values of k that preserve reconstruction quality, the linear-to-MLP gap on both LAMs fails to close, and action Fisher does not meaningfully and consistently exceed the raw latent reference. The game label control confirms the tested SAE configurations work: the linear-to-MLP gap narrows at k=16 and k=32, and game Fisher climbs steeply with k, overtaking its raw-latent reference on both LAMs. This indicates the SAEs isolate game-selective structure when the information is present but merely superposed.Therefore, the lack of recovery of a comparably strong, stable, or linearly useful action-selective basis points to the conclusion that there is limited additional recoverable independent action information. This is consistent with action information with game- and state-dependent visual variation. VideoFlexTok shows a similar disparity (Appendix Table 11).

Table 2: **Q2: within-environment probe results.** Deeper probes decode actions well above baseline for every encoder, on both P2P and Retro. VideoFlexTok consistently achieves higher performance on Retro, although differences in latent dimensionality complicate direct comparison with the latent-action models.

	AdaWorld	Olaf-World	VideoFlexTok
P2P — linear probe			
Exact match acc.	26.0%	43.9%	–
Hamming loss	0.109	0.086	–
Micro F1	0.429	0.615	–
Macro F1	0.226	0.434	–
Exact match acc., MLP-2	56.8%	63.9%	–
Retro — action accuracy (majority 30.6–31.0%)			
Linear	45.1%	48.7%	65.6%
MLP-1	57.5%	64.5%	82.1%
MLP-2	70.4%	73.9%	**82.2%**
Retro — game accuracy (193 classes)			
Linear	75.8%	83.8%	99.2%
MLP-2	98.2%	98.4%	99.2%
Null transition, MLP-2	98.7%	98.8%	99.2%

4 Q2: Is action information decodable into actions inside a known environment?

Nonlinearity and probe depth improve action decoding beyond what feature-axis alignment alone provides. (Table 2). On P2P (multi-label, 10 keys) Olaf-World roughly doubles AdaWorld's exact-match accuracy and leads on every F1 aggregate; on Retro all three encoders clear the majority-class baseline. Depth improves every one of these numbers, as Section 3 anticipates: no single axis separates the classes, so a nonlinear boundary can use structure that a linear one cannot. The answer to Q2 is therefore yes at every depth we measure, but only as decodability inside environments the probe was fitted on. These pooled probes can also use game identity, so this result does not show a mapping from latents to actions that is shared across games (see below and Q3).

Accuracy varies widely across keys and across games. Each number in Table 2 is an average, over the ten keys on P2P and over the 193 games on Retro, and accuracy is not uniform along either axis. On P2P, per-key F1 broadly follows label frequency: the rarest keys have near-zero F1 under the linear probes (Appendix Table 14). Deeper probes, however, improve F1 for every key, including most of those that the linear probes fail to decode. Thus, near-zero linear F1 indicates a limitation of the probe, not that the corresponding key is absent from the latent. The same variation appears across Retro games. Probes fitted on a single game cannot use game identity, and still reach mean accuracies of 0.48–0.59 (linear) and 0.70–0.76 (MLP-2) for the two LAMs. Per-game accuracy ranges from the game's majority-class baseline to perfect for every encoder (Appendix C.1). For AdaWorld, a sizeable minority of games are at or below their baseline with a linear probe, compared with only one at MLP-2. The average score reported across all games therefore hides substantial differences in how easily actions can be decoded from individual games.

Pooled decoding does not imply a shared action representation. The per-game results are consistent with two possibilities: different games may require different mappings from latent to action, or a shared mapping may simply be harder to decode in some games. The pooled probe cannot distinguish these explanations because it is trained on all 193 games, and game identity itself is linearly decodable (99.2% for VideoFlexTok and 75.8–83.8% for the LAMs; Table 2). Figure 3 shows the same separation geometrically: latents cluster by game, while action labels do not occupy consistent regions across those clusters. The pooled probe can therefore exploit game-specific information when predicting actions. We test whether action decoding generalizes across games in Section 5.

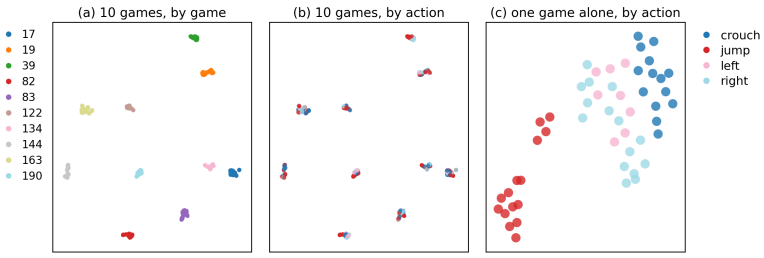


Figure 3: **Actions separate inside a game and not across games.** UMAP plots of VideoFlexTok latents on the Retro dataset. (a) and (b) are the same embeddings colored twice: the ten islands of (a) are the ten games, and in (b) the same islands carry the four action colors, with no color holding a region of its own across the plot. (c) fixes one specific game, and actions are easier to separate. This is evidence that latents mostly encode the scene, and that latent action information is only a small, strongly entangled residual. Additional global and per-game latent projections are provided in Appendix D.3

Table 3: **A probe's accuracy inside its training games does not transfer to games it has never seen.** One fitted probe per row, scored on two test sets: *Within*, the 20% of samples held back from the 154 training games, and *Cross*, every sample from the 39 games held out entirely. Δ is the cost of never having seen the environment. Deep MLP classifiers specialize for specific games, and do not transfer reliably to unseen ones.

	Action accuracy (%)		
	Within (154 games)	Cross (39 unseen)	Δ
Linear probe			
AdaWorld	45.3	42.2	-3.0
Olaf-World	49.5	45.0	-4.5
VideoFlexTok	66.5	53.2	-13.3
MLP-2 probe			
AdaWorld	71.1	40.8	-30.3
Olaf-World	74.5	41.3	-33.1
VideoFlexTok	82.6	50.9	-31.7

5 Q3: Do action predictors trained in known environments transfer to new environments?

Nonlinear probe capacity buys accuracy inside the training environments and costs it outside them. Replacing the linear probe with MLP-2 adds tens of points inside the training games, for both LAMs, and subtracts points on the held-out games, for every encoder (Table 3). This sign flip, together with the finding that action information is entangled with game identity (Table 2), suggests that the MLP is exploiting nonlinear, game-conditioned associations between representation and controller label rather than learning an environment-invariant action rule: such associations help once the probe has already seen the relevant games, but carry over poorly to games it has not. The linear probe scores lower within-environment (Table 2), but its weaker signal transfers more reliably and stays above chance in unseen environments.

The failure is spread across the held-out games, not concentrated in a few. Table 4 breaks the MLP-2 cross cell out by test game: the median sits beside the mean and the range, though wide, is not bimodal. The pooled cross accuracy is therefore not an average over successful transfers and failures: it is close to what nearly every held-out game gives on its own. The probe does beat most held-out games' own majority class, so the mapping carries information; but this is not a usable control signal.

Entanglement points the same way. Every Retro ratio exceeds 1 (Table 4): changing the game moves the representation substantially more than changing the action, and it does so for the tokenizer as much as for the two LAMs. On P2P, AdaWorld sits at parity and Olaf-World marginally below, consistent with the corpus difference (a P2P action produces broadly the same optical signature in

Table 4: **The cross-environment gap is spread evenly across held-out games, not driven by a handful of outliers.** This is the *same* probe as the MLP-2 cross cell of Table 3, tested across the 39 held-out games. The per-game means reproduce the pooled cross accuracies there to within 0.5 points. *Fraction above baseline* is the share of the 39 games on which the probe beats that game's own majority class. *Entanglement ratio* is the mean pairwise L2 distance of (same action, different game) over (different action, same game), measured on the full corpus. A ratio above 1 means changing the game moves the representation more than changing the action: games determine latents' coordinates more than actions do.

	Mean	Min–Max	Median	Fraction above baseline	Entanglement ratio
AdaWorld	40.8	23.4–91.1	36.9	0.85	1.18
Olaf-World	41.5	23.4–81.1	39.3	0.85	1.34
VideoFlexTok	51.3	28.7–96.4	49.6	0.95	1.38

every game, whereas the Retro controls drive different sprites, physics and scroll behavior in each of the 193 titles).

6 Discussion

Actions are decodable within environments, but the mapping does not transfer. Action labels can be predicted from all three representations (Q1), and decoded wherever the probe has seen the environment (Q2), but a single fitted latent-to-action mapping does not carry well to an environment it was not fitted on (Q3), especially for the nonlinear probe. Decodable action information in these representations is therefore environment-specific, consistent with the entanglement evidence.

Probe capacity makes a representation look more transferable than it is. Depth and nonlinearity add tens of points inside the fitted environments and subtract points outside them (Table 3), so the extra capacity buys game-conditioned associations rather than an environment-invariant action rule (entanglement ratios and per-game breakdown of Table 4 independently support this claim). Two consequences follow. First, evaluation on held-out samples from known environments overstates the quality of a representation as a transferable action interface. Second, the linear probe is a more honest estimate of decodability of the latents in a new environment (across all probes): above chance but far below maximum within-environment numbers. If transferable shared action structure corresponds with linearly decodable action information, the inability of the top-k SAEs studied to recover additional linearized action-related structure could help explain why nonlinear probes do not outperform linear probes in unseen environments.

The result does not depend on which LAM is stronger. Olaf-World outperforms AdaWorld on every probe experiment, yet its entanglement ratio is the higher of the two (game identity dominates action *more* in its latent by that measure), even as its accuracy and Fisher separability are both higher. Moreover, Olaf-World's MLP-2 cross-environment drop is the larger of the two in absolute terms (Table 3), so better within-environment decodability does not guarantee better transfer.

The best within-environment encoder is not a LAM. VideoFlexTok has no inverse-dynamics objective and no action supervision of any kind, yet reads Retro actions better than either LAM at every probe depth (Table 2). The comparison confounds four factors at once (training objective, latent width, input window, and training corpus) and a width-only control that truncates the latent along the token axis (Appendix C.2) narrows the gap, so we report the advantage as an observation rather than an architectural result. A natural next step is to train a LAM with VideoFlexTok's latents as alignment target, in place of Olaf-World's V-JEPA 2 teacher.

Limitations. We study two closely related LAMs, AdaWorld and Olaf-World, so our conclusions are specific to these models and their learned representations. We deliberately evaluate across video games to test whether latent action representations transfer across games; whether the same findings hold in other action domains, such as robotics, remains to be tested. In Retro, action labels need not correspond to the same in-game behavior across games or within-game states. For example, a label such as `jump` does not necessarily produce the same visual transition in every example.

Consequently, each action class can contain heterogeneous visual effects, adding variability within the class. Moreover, our Retro controller holds one action for 4 frames after 16 idle frames, which makes the action and its visual effect strongly correlated by design. Our controls remove the transition (null pairs) but do not separate the action from its visual consequence: a probe may read the visual change rather than an action code. Controls that mask or perturb action-correlated visual cues are left to future work. A separate limitation is that our default frame-pair construction uses class-dependent spacing, which can provide an additional cue to the probes. The cross-game drop persists under a fixed-spacing control for both LAMs (Appendix Table 6), suggesting that this cue does not account for the transfer gap. Because the fixed-spacing control changes the evaluated frame pairs, it does not isolate frame spacing from differences in visual content.

7 Related Work

World Models and Latent Actions. World models learn predictive representations of environment dynamics. Early world models combined compact visual representations with action-conditioned latent dynamics from interaction trajectories [Ha and Schmidhuber, 2018]. Latent-action methods relax the need for recorded actions by inferring latent control variables directly from visual transitions. LAPO [Schmidt and Jiang, 2024] infers latent actions and learns predictive dynamics from trajectories without action labels, and uses the learned representations for policy pretraining. Genie [Bruce et al., 2024] learns discrete latent actions from video to condition an interactive generative world model. AdaWorld [Gao et al., 2025] uses continuous latent actions to condition a pretrained world model. Olaf-World [Jiang et al., 2026] focuses on aligning continuous latent actions across visual contexts. More recent work explores joint latent-action and world-model training in CoLA-World [Wang et al., 2025] and factorized latent actions for multi-entity dynamics in FLAM [Wang et al., 2026b].

Understanding Latent Action Spaces. Recent work examines whether latent actions encode controllable actions or more general sources of visual change. Zhang et al. [2025] show that, under a linear model, latent-action learning can recover principal directions from a mixture of controllable state changes and exogenous variation, so the learned representation need not correspond to action alone. Empirically, Garrido et al. [2026] find that latent actions learned from in-the-wild video can capture spatially localized, camera-relative transformations in the absence of a shared embodiment. Olaf-World [Jiang et al., 2026] also evaluates cross-context alignment of latent actions through linear probing.

Video Tokenizers. Video tokenizers compress videos into representations suitable for downstream generative modeling. Many approaches encode videos into fixed spatiotemporal grids of tokens using VAE- or VQ-VAE-based architectures [van den Oord et al., 2017]. VideoFlexTok [Atanov et al., 2026] uses video VAE latents and resamples them into a variable-length, coarse-to-fine token sequence, with earlier tokens capturing higher-level semantic information and later tokens progressively adding visual detail. Unlike the latent-action models studied above, VideoFlexTok is trained to represent visual content rather than to infer latent actions. We include it as a visual-representation baseline to test whether action information observed in LAM latents is specific to latent-action learning or can also emerge in a general-purpose video representation learned without an action-related objective.

8 Conclusion

We investigated whether pretrained latent action models learn action representations that remain consistent across environments. Action labels are clearly decodable within environments seen during fitting, but the same latent-to-action mapping generalizes poorly to unseen ones. Increasing probe capacity improves within-environment decoding far more than cross-environment transfer. Thus, the limitation is not that actions cannot be decoded, but the lack of a stable, environment-independent mapping between latent representations and actions. A general-purpose video tokenizer shows the same pattern while providing the strongest within-environment action signal of any representation we evaluate. Together, these results show that strong action decodability does not by itself imply a transferable action representation. Understanding which learning objectives and architectural choices promote this consistency remains an important open question.

Acknowledgments and Disclosure of Funding

The authors received no specific funding for this work and declare no competing interests.

References

Andrei Atanov, Jesse Allardice, Roman Bachmann, Oğuzhan Fatih Kar, R Devon Hjelm, David Griffiths, Peter Fu, Afshin Dehghan, and Amir Zamir. Videoflextok: Flexible-length coarse-to-fine video tokenization, 2026. URL https://arxiv.org/abs/2604.12887.

Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

Shenyuan Gao, Siyuan Zhou, Yilun Du, Jun Zhang, and Chuang Gan. Adaworld: Learning adaptable world models with latent actions, 2025. URL https://arxiv.org/abs/2503.18938.

Quentin Garrido, Tushar Nagarajan, Basile Terver, Nicolas Ballas, Yann LeCun, and Michael Rabbat. Learning latent action world models in the wild, 2026. URL https://arxiv.org/abs/2601.05230.

David Ha and Jürgen Schmidhuber. World models, 2018.

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels, 2019. URL https://arxiv.org/abs/1811.04551.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2023.

Yuxin Jiang, Yuchao Gu, Ivor W. Tsang, and Mike Zheng Shou. Olaf-world: Orienting latent actions for video world modeling, 2026. URL https://arxiv.org/abs/2602.10104.

Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models, 2022.

Mathieu Poliquin. Stable retro, a maintained fork of OpenAI's gym-retro. https://github.com/Farama-Foundation/stable-retro, 2026.

Nedko Savov, Naser Kazemi, Mohammad Mahdi, Danda Pani Paudel, Xi Wang, and Luc Van Gool. Exploration-driven generative interactive environments. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pages 27597–27607, June 2025.

Dominik Schmidt and Minqi Jiang. Learning to act without actions. In *International Conference on Learning Representations*, volume 2024, pages 9379–9395, 2024.

Aaron van den Oord, Oriol Vinyals, and koray kavukcuoglu. Neural discrete representation learning. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/7a98af17e63a0ac09ce2e96d03992fbc-Paper.pdf.

Yucen Wang, Fengming Zhang, De-Chuan Zhan, Li Zhao, Kaixin Wang, and Jiang Bian. Co-evolving latent action world models. *arXiv preprint arXiv:2510.26433*, 2025.

Zhengqing Wang, Saurabh Nair, Prajwal Chidananda, Pujith Kachana, Samuel Li, Matthew Brown, and Yasutaka Furukawa. La-pose: Latent action pretraining meets pose estimation, 2026a. URL https://arxiv.org/abs/2604.27448.

Zizhao Wang, Chang Shi, Jiaheng Hu, Kevin Rohling, Roberto Martín-Martín, Amy Zhang, and Peter Stone. Factored latent action world models. *arXiv preprint arXiv:2602.16229*, 2026b.

Seonghyeon Ye, Joel Jang, Byeongguk Jeon, Sejune Joo, Jianwei Yang, Baolin Peng, Ajay Mandlekar, Reuben Tan, Yu-Wei Chao, Bill Yuchen Lin, et al. Latent action pretraining from videos, 2024.

Yuguang Yue, Irakli Salia, Samuel Hunt, Chris Green, Wenzhe Shi, and Jonathan J Hunt. Scaling behavior cloning improves causal reasoning: An open model for real-time video game playing, 2026. URL https://arxiv.org/abs/2601.04575.

Chuheng Zhang, Tim Pearce, Pushi Zhang, Kaixin Wang, Xiaoyu Chen, Wei Shen, Li Zhao, and Jiang Bian. What do latent action models actually learn?, 2025. URL https://arxiv.org/abs/2506.15691.

Appendix

Section A describes the encoders, the two corpora, the frame spacing, the P2P subset, and the held-out-game split of Section 5; Section A.6 names all 193 Retro games. Section B is the sparse-basis control of Q1. Section C gives the per-game, per-key and per-width detail of Q2. Section D collects the geometry, projection and visual-content analyses, which characterize the latent spaces but do not directly answer any level of Q1–Q3.

A Corpora, encoders, and protocol

A.1 Implementation details

Table 5 collects the configurations used in our experiments. These details are provided for reproducibility.

Table 5: **Model and extraction configurations.** Widths are the latent representations used by the probes: 32 for AdaWorld and Olaf-World, and either the 32×6 token prefix or the full 256×6 sequence for VideoFlexTok. The evaluated AdaWorld and Olaf-World checkpoints use the same encoder/decoder depths, channel width, and number of attention heads; Olaf-World uses causal temporal attention in its encoder. Its Retro inputs are evaluated at 256×256, below the checkpoint's training resolution of 272×480.

	AdaWorld	Olaf-World	VideoFlexTok
Checkpoint	`Little-Podi/AdaWorld`	`YuxinJ/Olaf-World`	`EPFL-VILAB/videoflextok_d18_d28`
Parameters	$\sim$500M	$\sim$500M	286.7M
Blocks	16 enc. + 16 dec.	16 enc. + 16 dec.	18
Channels	1024	1024	—
Attention heads	16	16	—
Bottleneck	VAE	VAE	FSQ, 6-d code, 64,000 entries
Latent width	32	32	192 (32×6) or 1536 (256×6)
Input frames	2	2	17
Retro input res.	256×256	256×256	256×256
P2P input res.	$640 \times 480 \rightarrow 256 \times 256$	272×480	—

Checkpoints: huggingface.co/Little-Podi/AdaWorld, huggingface.co/YuxinJ/Olaf-World, huggingface.co/EPFL-VILAB/videoflextok_d18_d28. For VideoFlexTok we read the continuous pre-quantization FSQ vectors, so the codebook size affects the encoder's training but not the vectors we probe.

A.2 Retro corpus construction

Frames are collected with the `random_action_agent` of GenieRedux [Savov et al., 2025], sampling the 20-frame cycle of Figure 4: the agent idles for 16 frames and then holds one randomly chosen action for the final 4, so that a single action's visible effect is isolated per cycle and the encoders are given a clean signal to encode. The 193 game ids span 98 NES, 63 SNES, 24 Master System, 7 Game Boy and 1 Atari 2600 title. The corpus was hand-curated: the collection script skips a ROM only when *none* of the four action buttons maps to anything in that game's action space, so it does not by itself guarantee that all four are present, and the 193 titles were selected by hand from what it produced. Cycles whose sampled button has no mapping in a given game are emitted without an action label and are excluded from every probe. What this rule does *not* guarantee is that a mapped button has the visual effect its name implies, and we do not measure that agreement.

A.3 How far apart the two frames are

The pair handed to AdaWorld and Olaf-World *brackets* the action's visible consequence rather than straddling a single step, so the intermediate trajectory is never shown. VideoFlexTok needs 17 frames to match its $4\times$ temporal compression, so its window extends into the genuine frames preceding and following the action window, and it is *not* positioned identically for every class: `jump` windows are

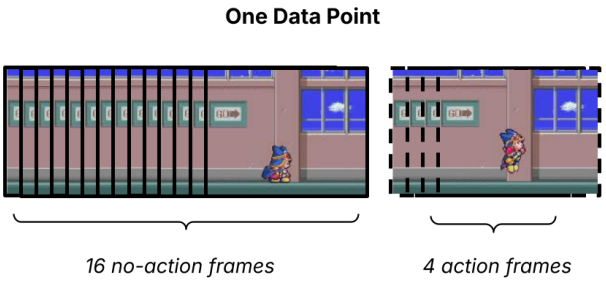


Figure 4: Custom sampling mechanism for the Retro dataset.

advanced by 10 frames so they contain the jump's visible consequence, while the other three are contiguous.

How long a press takes to show on screen differs by action, so the two frames are spaced accordingly: 11 frames apart for `jump`, which has to leave the ground and land again, 7 for `right` and `left`, and 4 for `crouch`. Every Retro number in the main text uses this per-action spacing. Spacing that depends on the label is a cue in its own right, and changing it changes *which* pairs exist, not merely how they are labeled. The worry is therefore that the paper's central claim, that a decoder which works within an environment does not transfer to a new one, is an artifact of the spacing we happened to run. We re-ran both LAMs, on both splits and at all three depths, with a fixed 4-frame spacing for every action; Table 6 compares the two.

Table 6: Change in action accuracy on the held-out games of Q3 (percentage points) under each frame spacing. All changes are negative for both encoders, so the failure to transfer does not depend on the spacing.

	spacing	linear	MLP-1	MLP-2
AdaWorld	fixed	−2.03	−11.90	−23.90
	per-action	−2.81	−13.31	−29.81
Olaf-World	fixed	−1.68	−13.70	−25.12
	per-action	−3.71	−17.23	−32.05

A.4 P2P corpus composition

Table 7 gives the label and game distribution of the P2P subset described in Section 2.2, against which the per-key F1 columns of Table 14 are read. The ten keys are `w`, `a`, `s`, `d`, the four arrow keys, `LeftShift`, `f` and `none`, and a frame pair may carry any subset of them.

In this dataset, a resolution filter corresponds to a game filter. That is why the composition is reported at all: this subset is not interchangeable with a differently filtered one drawn from the same release.

Table 7: P2P corpus composition, full subset B (999,720 samples over 291 sessions and 15 games, mean 1.31 keys held per sample). `w` dominates at 42.8%, and the game distribution is highly concentrated: the top three games account for 68% of samples. This skew is why the per-key results of Table 14 describe this particular subset rather than P2P as a whole.

Key	share of samples	Game	samples	share
`w`	42.76%	be-a-snake	264,206	26.43%
`none`	22.54%	blade-ball	264,036	26.41%
`d`	16.22%	death-ball	155,358	15.54%
`a`	13.89%	blade-ball-elemental	68,409	6.84%
`UpArrow`	10.54%	be-a-dragon	53,697	5.37%
`s`	7.56%	be-a-shark	53,265	5.33%
`LeftArrow`	7.20%	be-a-tornado	43,766	4.38%
`RightArrow`	7.01%	unknown	23,541	2.35%
`LeftShift`	2.57%	be-a-alligator	18,295	1.83%
`f`	0.88%	be-a-trex	15,863	1.59%
		nature-disaster-survival	14,800	1.48%
		bear-simulator	10,858	1.09%
		roblox-rally-racing	7,737	0.77%
		snake-simulator	5,448	0.54%
		jujutsu-shenanigans	441	0.04%

A.5 The held-out-game split

The cross-environment split of Section 5 partitions the 193 Retro games, not the samples. 39 games are drawn once from the game list and contribute *no* training samples to any probe. The probe of Table 3 is fitted once, on a random 80% of the labeled samples of the 154 training games, and then scored twice: on the 20% of those samples it held back (*within*) and on every labeled sample of the 39 held-out games (*cross*). The two columns share one set of weights, so there is no difference in training corpus, training-set size, sample order, or initialization.

Table 8: Sample counts and majority baselines for both test sets. The two LAMs share a sample set; VideoFlexTok's is smaller because its 17-frame input window yields fewer windows per clip. Class frequencies are of the test set named in the row.

	train	test	right	left	crouch	jump
AdaWorld and Olaf-World						
within	302,677	75,669	31.1	15.2	23.0	30.8
cross	302,677	101,942	29.5	14.8	26.2	29.5
VideoFlexTok						
within	281,858	70,465	30.8	15.4	23.0	30.8
cross	281,858	95,264	29.4	14.9	26.3	29.5

Table 8 gives the sample counts and class frequencies of both test sets. The two differ by at most 1.6 points of majority baseline, so Δ in Table 3 is not a change in how hard the 4-way problem is: `crouch` is 3 points commoner in the held-out games and `right` 1.6 points rarer, which moves the majority class but not the difficulty.

A.6 The 193 Retro games

Table 9 names every game in the Retro corpus.

Table 9: The 193 stable-retro games of the Retro corpus: 98 NES, 63 SNES, 24 Master System, 7 Game Boy and 1 Atari 2600 title. The 39 marked † are the held-out games of Section A.5, which contribute no training sample to any probe; the other 154 are the training games. All 193 contribute to the pooled analyses of Section 3 – Section 4.

1 8Eyes-Nes	2 ActionPachio-Snes	3 AddamsFamily-Nes
4 AdventureIsland-GameBoy	5 AdventureIsland3-Nes	6 AdventureIslandII-Nes
7 AdventuresOfDrFranken-Snes	8 AdventuresOfKidKleets-Snes	9 AdventuresOfMightyMax-Snes
10 AdventuresOfRockyAndBullwinkleAnd Friends-Nes	11 AdventuresOfRockyAndBullwinkleAnd Friends-Snes	12 AdventuresOfStarSaver-GameBoy

Table 9, continued from the previous page.

13 AdventuresOfYogiBear-Snes†	14 AeroTheAcroBat-Snes†	15 AeroTheAcroBat2-Snes
16 AlfredChicken-GameBoy†	17 AlfredChicken-Nes	18 Alien3-Sms†
19 Amagon-Nes	20 Asterix-Sms	21 Asterix-Snes
22 AsterixAndObelix-GameBoy	23 AsterixAndTheSecretMission-Sms†	24 Astyanax-Nes
25 Athena-Nes	26 AtlantisNoNazo-Nes	27 AttackOfTheKillerTomatoes-Nes
28 BanishingRacer-GameBoy	29 Barbie-Nes	30 BartSimpsonsEscapeFromCampDeadly-GameBoy
31 BioMiracleBokutteUpa-Nes	32 BombJack-GameBoy	33 BoogermanAPickAndFlickAdventure-Snes†
34 BramStokersDracula-Snes	35 BronkieTheBronchiasaurus-Snes	36 BubbleBobble-Nes
37 BubbleBobble-Sms	38 BubsyII-Snes	39 BugsBunnyBirthdayBlowout-Nes
40 Castlevania-Nes	41 CastlevaniaIIIDraculasCurse-Nes	42 CatNindenTeyandee-Nes
43 ChesterCheetahTooCoolToFool-Snes†	44 ChuckRock-Sms	45 ChuckRock-Snes
46 CircusCharlie-Nes	47 Cliffhanger-Nes	48 ConquestOfTheCrystalPalace-Nes
49 CyberShinobi-Sms	50 DananTheJungleFighter-Sms	51 Darkman-Nes
52 DeepDuckTroubleStarringDonaldDuck-Sms	53 DennisTheMenace-Snes	54 DiggerTheLegendOfTheLostCity-Nes†
55 DinoCity-Snes	56 DonDokoDon-Nes	57 DonkeyKongCountry2-Snes
58 DonkeyKongCountry3DixieKongsDoubleTrouble-Snes	59 DonkeyKongJr-Nes	60 DragonsLair-Snes
61 FamilyDog-Snes	62 FelixTheCat-Nes	63 FinalBubbleBobble-Sms
64 FirstSamurai-Snes	65 FormationZ-Nes	66 FoxsPeterPanAndThePiratesTheRevengeOfCaptainHook-Nes†
67 GhostsnGoblins-Nes†	68 GhoulSchool-Nes†	69 GIJoeARealAmericanHero-Nes
70 GIJoeTheAtlantisFactor-Nes	71 Gimmick-Nes	72 GreatCircusMysteryStarringMickeyAndMinnie-Snes
73 HarleysHumongousAdventure-Snes†	74 Hook-Snes	75 IceClimber-Nes
76 IncredibleHulk-Sms	77 InspectorGadget-Snes	78 IronSwordWizardsAndWarriorsII-Nes
79 JackieChansActionKungFu-Nes†	80 JajamaruNoDaibouken-Nes	81 JetsonsCogswellsCaper-Nes
82 JetsonsInvasionOfThePlanetPirates-Snes†	83 JoeAndMac-Nes	84 JoeAndMac-Snes†
85 JourneyToSilius-Nes	86 JungleBook-Nes	87 KabukiQuantumFighter-Nes†
88 KaiketsuYanchaMaru2KarakuriLand-Nes†	89 KaiketsuYanchaMaru3TaiketsuZouringen-Nes	90 KamenNoNinjaAkakage-Nes
91 KidKlownInNightMayorWorld-Nes	92 KidNikiRadicalNinja-Nes	93 LandOfIllusionStarringMickeyMouse-Sms
94 LastActionHero-Snes	95 LowGManTheLowGravityMan-Nes	96 LuckyDimeCaperStarringDonaldDuck-Sms
97 MagicalQuestStarringMickeyMouse-Snes†	98 MagicBoy-Snes	99 MagicSword-Snes
100 MarioBros-Nes	101 Mask-Snes	102 MasterOfDarkness-Sms†
103 MCKids-Nes	104 MegaMan-Nes	105 MegaMan2-Nes†
106 MetalStorm-Nes	107 MickeyMousecapade-Nes	108 MightyBombJack-Nes
109 MitsumeGaTooru-Nes	110 MonsterInMyPocket-Nes	111 MontezumaRevenge-Atari2600
112 NewZealandStory-Sms	113 NinjaCrusaders-Nes†	114 NinjaGaiden-Nes
115 NinjaGaiden-Sms	116 NinjaGaidenIIITheAncientShipOfDoom-Nes†	117 NinjaGaidenIITheDarkSwordOfChaos-Nes
118 NinjaKid-Nes	119 OutToLunch-Snes	120 PacInTime-Snes
121 PanicRestaurant-Nes†	122 PitfallTheMayanAdventure-Snes†	123 PizzaPop-Nes†
124 Plok-Snes	125 PorkyPigsHauntedHoliday-Snes	126 PowerPiggsOfTheDarkAge-Snes
127 PsychicWorld-Sms	128 PuttySquad-Snes†	129 RadicalRex-Snes
130 RainbowIslands-Nes	131 RainbowIslandsStoryOfTheBubbleBobble2-Sms	132 Realm-Snes
133 RenAndStimpyShowVeediots-Snes	134 RoadRunnersDeathValleyRally-Snes†	135 RoboccoWars-Nes
136 RoboCop2-Nes	137 RoboCop3-Nes	138 RoboCop3-Sms
139 RoboCopVersusTheTerminator-Sms†	140 RunningBattle-Sms	141 RunSaber-Snes†
142 SDHeroSoukessenTaoseAkuNoGundan-Nes	143 SeikimaIIAkumaNoGyakushuu-Nes	144 Shatterhand-Nes
145 SimpsonsBartVsTheSpaceMutants-Nes	146 SimpsonsBartVsTheWorld-Nes†	147 SkuljaggerRevoltOfTheWesticans-Snes
148 Smurfs-Nes	149 Smurfs-Snes	150 SnowBrothers-Nes†
151 SpankysQuest-Snes	152 StarWars-Nes	153 SuperAdventureIsland-Snes
154 SuperBCKid-Snes	155 SuperCastlevaniaIV-Snes	156 SuperGhoulsnGhosts-Snes
157 SuperJamesPond-Snes	158 SuperMarioBros-Nes	159 SuperMarioBros3-Nes†
160 SuperPitfall-Nes	161 SuperStarWars-Snes†	162 SuperStarWarsTheEmpireStrikesBack-Snes†
163 SuperTurrican-Snes	164 SuperTurrican2-Snes†	165 SuperWidget-Snes†
166 SuperWonderBoy-Sms	167 SwampThing-Nes	168 TazMania-Sms†
169 TeenageMutantNinjaTurtles-Nes	170 Terminator2JudgmentDay-Nes	171 TetsuwanAtom-Nes
172 TimeZone-Nes	173 TinyToonAdventures-Nes	174 TinyToonAdventuresBusterBustsLoose-Snes
175 Toki-Nes†	176 TomAndJerry-Snes	177 TotallyRad-Nes
178 TotalRecall-Nes†	179 TreasureMaster-Nes	180 TrollsInCrazyland-Nes
181 UruseiYatsuraLumNoWeddingBell-Nes	182 WaynesWorld-Nes	183 WereBackADinosaursStory-Snes
184 Widget-Nes	185 WizardsAndWarriors-Nes	186 Wolfchild-Snes
187 WonderBoyInMonsterWorld-Sms	188 WrathOfTheBlackManta-Nes	189 XMenMojoWorld-Sms
190 YoukaiClub-Nes	191 YoukaiDouchuuki-Nes	192 YoungIndianaJonesChronicles-Nes
193 ZeroTheKamikazeSquirrel-Snes		

B Q1: Sparse autoencoders

This section describes the sparse-basis control of Section 3: implementation, training, and detailed results.

B.1 Implementation

We implement SAEs with a single hidden layer encoder and a linear decoder. The encoder applies a linear transformation followed by a ReLU non-linearity to map the original latents into a higher-dimensional feature space, and the decoder is a linear layer that reconstructs the original latents from these sparse activations. During training, we apply a top-k mask to the encoder activations (keeping only the k largest values per example) and train the decoder to reconstruct the original

latents from these masked activations using mean squared error. The input mean is subtracted before encoding and added back after decoding. This matters most at $k = 1$, where disabling it drops reconstruction explained variance from 0.542 to 0.184 and $k = 1$ game accuracy from 94.3% to 26.8% on VideoFlexTok. Additionally, decoder columns are unit-normalized at initialization.

SAEs were trained with $k = 1, 2, 4, 8, 16, 32, 64$. Feature dimensionality was set to $F = 256$ for AdaWorld and Olaf-World (32-dimensional latents) and $F = 512$ for VideoFlexTok (192-dimensional latents) to allow an over-complete sparse basis in each case. We also ran a VideoFlexTok sweep at $F = 12{,}288$ over the full 1536-dimensional latent; see the note under Table 10.

B.2 Detailed SAE results

SAEs make game identity coordinate-aligned but do not recover a comparably sparse action basis.

On the two LAMs, game accuracy is poor when only one feature may fire and climbs steeply with k, overtaking the raw-latent reference in both cases (Figure 6, Figure 5). Useful game information therefore comes from a *combination* of units rather than a single "game neuron". VideoFlexTok is already at 94.3% with one feature and saturates by $k = 16$, so there is no climb left to observe (Figure 7). The two per-feature statistics agree. Game correlations rise above their raw references on all three encoders, and game Fisher scores meet or exceed raw at three or four of seven k on the LAMs (Figure 2) and exceed it by a factor of 351 on VideoFlexTok (Table 11).

Action Fisher stays near zero throughout on Retro (peaking at 0.132 across encoders, against game scores three to four orders of magnitude larger). This indicates action classes overlap along every single SAE dimension. The one statistic that moves is correlation: nothing exceeds 0.34, but the top-10 mean roughly triples on both LAMs and finishes above raw, so sparsity does redistribute what little action correlation exists. The effect is magnified for VideoFlexTok: correlation is $\sim$1 everywhere for games, and Fisher scores increase by orders of magnitude, while action correlation and Fisher scores do not reach raw scores.

SAEs recover linear separability in games but not actions.

For game identity probing, SAEs at top-k 16 and 32 close the gap between linear and MLP-2 accuracy for AdaWorld and Olaf-World, with linear probing on the sparse code rising above the raw-latent accuracy score. This supports the finding from Table 11 that game identity can be reparameterized into explicit directions in the sparse space. Given the null-transition control of Section 3, in which game identity decodes at 98–99% on this corpus from every representation we hold (including the null-transition latents, Section 4), game identity information is already non-linearly separable from action information in the encodings. The effect of the SAE is therefore to linearize game identity information.

For actions, by contrast, SAE encodings change little. Accuracy is flat in k on all three encoders and never more than a fraction of a point above the raw latent (Table 10). On the LAMs the linear–MLP gap does *not* narrow for $k > 8$ (AdaWorld) and $k > 16$ (Olaf-World), even though explained variance approaches 1 and fewer features are dead. This indicates that the SAE is not linearizing an action signal.

P2P results. These findings do not extend to the P2P dataset. Running the same sweep on P2P, we find that action decoding, correlation and Fisher all rise above raw together and the action Fisher exceeds the game Fisher, reversing the Retro ordering. That reversal is not safe to lean on: `none` is one of the P2P action classes, so a single "no key held" feature can lift all three statistics at once. No conclusive claim about P2P can therefore be drawn from the sparse-basis results.

Summary. Part of the apparent game entanglement is an artifact of the coordinate system: a sparse reparameterization recovers game information the raw basis hides, for both LAMs on Retro and again on P2P (Table 10). In the same enlarged basis, linear action decoding and action Fisher scores on Retro are largely unchanged. For AdaWorld and Olaf-World, a possible reading is that the raw representations do not contain a linearly recoverable action structure to disentangle. In the case of VideoFlexTok, SAEs similarly do not discover additional structure, but the raw baseline has higher linear decodability and coordinate-aligned action structure.

B.3 The sweeps in numbers

This section includes additional plots and tables related to results discussed in Section B.2. The three figures below show the probing results; Table 10 is the tabular form, with two additional quantities: how many sparse encoding features die at each sparsity level, and how much of the latent the sparse code reconstructs. Higher reconstruction quality means the sparse code is more faithful to the encoder's latent. The number of dead features reflects the sparsity attained. Table 11 gives the per-feature correlation and Fisher scores behind Figure 2 and the VideoFlexTok statements above.

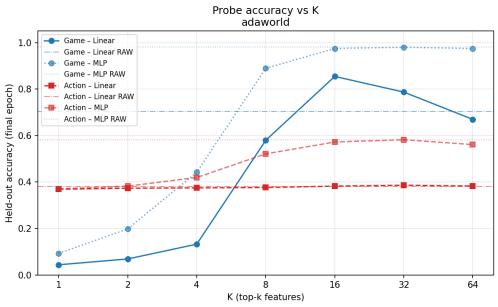


Figure 5: AdaWorld SAE probe accuracy per top-k ($F = 256$ over 32-d). The action linear/MLP gap *widens* with k towards the gap the raw latent already has.

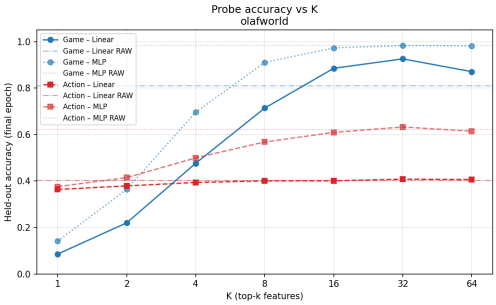


Figure 6: Olaf-World SAE probe accuracy per top-k ($F = 256$ over the 32-d latent); horizontal lines are the raw-latent references. Game accuracy climbs from 8.5% at $k = 1$ to 92.6% at $k = 32$, crossing its raw reference; action accuracy is flat on top of its own.

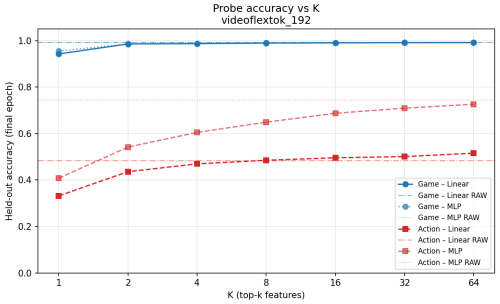


Figure 7: VideoFlexTok SAE probe accuracy per top-k at the configuration of Section B.1 ($F = 512$ over 192-d). Action accuracy meets and then exceeds its 192-d raw reference from $k = 8$, so the sparse code costs no action signal at this width. Unlike the two LAMs it shows no game climb, because it starts saturated at 94.3% with a single feature.

Table 10: For all encoders, as k increases, the number of dead features decreases, and the explained variance (reconstruction quality) increases. The many dead features for AdaWorld and Olaf-World, even at higher k, should be read against the low rank of the original latents (Appendix D.1). VideoFlexTok at $k \geq 8$ has no dead features. However, its reconstruction quality remains lower than for the other two encoders, which reach explained variance of 1.0 at higher k. A possible explanation is that using only the first 32×6 dimensions (rather than all 256×6) loses information and reduces learnability. The *raw* rows here use a different probe architecture from Table 2: Adam at 50 epochs, depths 0 and 3, so the "MLP" column is MLP-3, whereas Table 2 uses an sklearn logistic probe for its linear column. The SAE rows also use different samples: they were fitted at fixed frame spacing and not re-fitted at per-action spacing (Section A.3). Neither difference affects the claims, since the raw and SAE rows of this table share the same probe architecture and samples.

Encoder	k	dead	expl. var.	action lin.	action MLP	game lin.	game MLP
AdaWorld — dictionary 256 over 32-d, 604,194 samples							
	1	248	0.939	0.369	0.370	0.044	0.093
	2	243	0.988	0.373	0.382	0.069	0.198
	4	236	0.999	0.375	0.419	0.132	0.442
	8	232	1.000	0.376	0.521	0.580	0.889
	16	214	1.000	0.382	0.572	0.854	0.974
	32	190	1.000	0.386	0.582	0.787	0.980
	64	168	1.000	0.383	0.561	0.670	0.974
	raw	—	—	0.381	0.582	0.704	0.981
Olaf-World — dictionary 256 over 32-d							
	1	228	0.759	0.364	0.376	0.085	0.142
	2	205	0.912	0.379	0.415	0.220	0.364
	4	193	0.957	0.394	0.500	0.476	0.696
	8	201	0.985	0.400	0.568	0.714	0.910
	16	195	0.997	0.401	0.609	0.885	0.973
	32	193	1.000	0.408	0.632	0.926	0.983
	64	179	1.000	0.406	0.614	0.871	0.981
	raw	—	—	0.402	0.621	0.810	0.984
VideoFlexTok — dictionary 512 over 192-d (32×6), 447,587 samples							
	1	277	0.542	0.332	0.408	0.943	0.955
	2	3	0.657	0.436	0.542	0.986	0.986
	4	4	0.695	0.469	0.605	0.987	0.987
	8	0	0.741	0.485	0.649	0.989	0.989
	16	0	0.794	0.496	0.687	0.991	0.990
	32	0	0.858	0.501	0.709	0.991	0.991
	64	0	0.927	0.515	0.726	0.992	0.991
	raw	—	—	0.483	0.743	0.991	0.991

(i) For AdaWorld and Olaf-World, game linear accuracy exceeds the raw linear score: 0.854 ($k = 16$) vs 0.704 for AdaWorld, 0.926 ($k = 32$) vs 0.810 for Olaf-World. (ii) For actions, there is no significant increase in linear probing accuracy for any k. (iii) Peak MLP game probe scores remain unchanged for all models. However, action MLP scores match or insignificantly exceed raw scores for both LAM models, but do not reach raw scores for VideoFlexTok (0.726 vs 0.743). (iv) VideoFlexTok, by contrast, shows no gain in game linear probing accuracy, but action linear accuracy rises to 0.515 ($k = 64$) from the raw 0.483.

A separate run at $F = 12{,}288$ over the full 1536-d latent never recovers its raw reference (0.654 against 0.660 linear, 0.770 against 0.822 MLP). A sparse bottleneck is not the cause because the two runs' raw references differ by 0.177 linear. Instead, the wider run is measuring the cost of the 32×6 truncation, applied to the SAE's input. The truncation results in under-used features (explained variance 0.781, roughly 7,000 of 12,288 features dead throughout) against 0.927 with zero dead features from $k = 8$ here.

Table 11: "top-10" is the mean over the ten highest-scoring features, "max" the highest-scoring feature; both are one-vs-rest, maximized over classes. The *raw* row is the same statistic on the unmodified latent, so it is constant across k. The raw game and raw action columns are not directly comparable, as four balanced action classes admit higher one-vs-rest correlations than 193 rare game classes.

		Pearson $\|r\|$				Fisher			
		game		action		game		action	
Encoder	k	top-10	max	top-10	max	top-10	max	top-10	max
AdaWorld — dictionary 256 over 32-d									
	1	0.115	0.234	0.063	0.232	0.16	0.71	0.011	0.057
	2	0.142	0.173	0.072	0.246	0.25	0.71	0.013	0.064
	4	0.218	0.300	0.085	0.238	0.49	0.71	0.014	0.060
	8	0.250	0.418	0.075	0.218	1.16	1.60	0.010	0.050
	16	0.258	0.295	0.090	0.190	1.13	1.55	0.011	0.038
	32	0.274	0.332	0.144	0.180	0.91	1.06	0.022	0.035
	64	0.202	0.359	0.165	0.181	0.92	1.25	0.029	0.035
	raw	0.163	0.179	0.138	0.210	0.76	1.05	0.026	0.069
Olaf-World — dictionary 256 over 32-d									
	1	0.410	0.694	0.060	0.253	0.45	1.01	0.009	0.068
	2	0.439	0.634	0.091	0.251	0.89	1.45	0.014	0.067
	4	0.344	0.411	0.110	0.252	1.13	2.32	0.018	0.068
	8	0.322	0.803	0.153	0.257	1.33	1.96	0.029	0.071
	16	0.280	0.460	0.143	0.252	1.13	2.14	0.027	0.068
	32	0.224	0.287	0.182	0.239	0.87	1.56	0.038	0.061
	64	0.231	0.301	0.162	0.211	1.08	1.51	0.030	0.047
	raw	0.168	0.231	0.140	0.215	1.03	1.63	0.027	0.053
VideoFlexTok — dictionary 512 over 192-d									
	1	0.999	0.999	0.050	0.061	588	751	0.003	0.004
	2	0.999	1.000	0.089	0.113	702	1294	0.009	0.015
	4	0.999	1.000	0.105	0.144	895	1506	0.013	0.021
	8	1.000	1.000	0.114	0.149	1249	2465	0.014	0.027
	16	1.000	1.000	0.113	0.158	1258	1786	0.014	0.026
	32	1.000	1.000	0.108	0.144	1126	1693	0.013	0.022
	64	0.999	1.000	0.114	0.295	729	1231	0.018	0.095
	raw	0.211	0.325	0.109	0.339	4.16	7.01	0.020	0.132

(i) Game correlations exceed their raw reference on all three encoders, but only VideoFlexTok stays constant at $\sim$1, exceeding raw 3–4$\times$. Olaf-World's peak max is 0.803 vs 0.238 (raw) and AdaWorld's is 0.418 vs 0.179 (raw). The increase carries over to top-10 (0.439 vs 0.168 and 0.274 vs 0.163). (ii) Action correlations are small for all encoders (≤ 0.34). Both models have peak correlation scores above raw for both top-10 and max. However, the increase is modest compared with games, at most 30%: from 0.14 to 0.182 for Olaf-World top-10. Top-10 rises with k on both LAMs (by 2.6$\times$ on AdaWorld and 3.0$\times$ on Olaf-World, from $k = 1$ to its peak), while max starts above raw at small k and then decreases for both. (iii) Action Fisher sits near zero for all encoders. (iv) By contrast, game Fisher increases above raw for all encoders, reaching 2465 for VideoFlexTok. The game Fisher scores peak at $k = 8$–16 for all models.

C Q2: decodability in detail

Section 4 reports one pooled probe per encoder, trained on all games.

C.1 Per-game and per-key probing

Table 12 fits one probe per Retro game and Table 13 does the same on P2P; Table 14 breaks the pooled P2P numbers out by key.

These tables add two readings to Section 4. First, Olaf-World is ahead of AdaWorld on both corpora and at every depth, a comparison the pooled numbers cannot make because each shows only one corpus. Second, the depth effect is encoder-specific: on the two LAMs a linear probe falls 0.17–0.22 short of MLP-2, while on VideoFlexTok the linear probe is already at its ceiling and depth buys nothing, since its per-game representations are separable without a nonlinearity.

Table 12: Per-game probing accuracy summary: 193 independent fits, one per game, trained and tested only on that game's own samples. For the Retro dataset on AdaWorld, Olaf-World and VideoFlexTok, accuracy is multiclass action accuracy. All cells use per-action frame spacing (Section A.3); MLP-3 is not reported here because it was not re-fitted at that spacing.

Model	Probe	Mean	Range
AdaWorld	Linear	0.48	0.24–1.00
AdaWorld	MLP-1	0.64	0.27–1.00
AdaWorld	MLP-2	0.70	0.26–1.00
Olaf-World	Linear	0.59	0.27–1.00
Olaf-World	MLP-1	0.73	0.42–1.00
Olaf-World	MLP-2	0.76	0.43–1.00
VideoFlexTok (256×6)	Linear	0.84	0.45–1.00
VideoFlexTok (256×6)	MLP-1	0.84	0.41–1.00
VideoFlexTok (256×6)	MLP-2	0.84	0.42–1.00
VideoFlexTok (32×6)	Linear	0.73	0.36–1.00
VideoFlexTok (32×6)	MLP-1	0.78	0.42–1.00
VideoFlexTok (32×6)	MLP-2	0.78	0.41–1.00

Table 13: Aggregate P2P probing performance, *averaged over the 15 environments*. For this reason, the exact-match column does not match the pooled figures of Table 2 (0.51 against 26.0%, 0.59 against 43.9%): a mean over games weights a 441-sample game equally with a 264,206-sample one. Hamming loss is over the 10 binary key decisions; a predictor that always answers "key not held" scores 0.13, which is why the exact-match column is the informative one.

Model	Probe	Exact Match	Hamming Loss
AdaWorld	Linear	0.51	0.07
AdaWorld	MLP-1	0.61	0.05
AdaWorld	MLP-3	0.66	0.05
Olaf-World	Linear	0.59	0.06
Olaf-World	MLP-1	0.68	0.04
Olaf-World	MLP-3	0.71	0.04

C.2 The VideoFlexTok token-prefix sweep

VideoFlexTok's action latent is 256 FSQ tokens $\times$ 6 channels laid out token-major, so a prefix of the 1536-d vector is exactly a prefix of the token sequence. We probe six prefix lengths (8/16/32/64/128/256 tokens).

Table 15 shows that action and game accuracy separate almost immediately. Table 16 shows that the Fisher jump between the two VideoFlexTok columns of Table 20 comes from a single coordinate. Table 17 shows the entanglement ratio of Table 20 falling monotonically with width, and Table 18 extends the per-game summary of Table 12 along the same ladder.

Table 15: Action and game probe accuracy (%) along the VideoFlexTok token prefix, on Retro. Game identity is already at 98.6% from 8 tokens and saturates; action accuracy keeps improving up to 256. The 32×6 truncation costs 17.5 points of linear action accuracy and 9.0 points at MLP-2.

		action accuracy				game accuracy	
tokens	dims	linear	MLP-1	MLP-2	MLP-3	linear	MLP-2
8	48	43.7	63.5	67.8	68.2	98.6	99.0
16	96	46.2	67.6	70.4	71.1	99.1	99.1
32	192	48.1	71.8	73.4	74.6	99.1	99.2
64	384	55.9	76.4	77.4	78.0	99.1	99.2
128	768	60.2	79.7	80.4	80.6	99.1	99.2
256	1536	65.6	81.9	82.4	82.2	99.2	99.2

Table 14: Per-key F1 on P2P, linear and MLP-2 probes (199,944 held-out frame pairs).

	AdaWorld		Olaf-World	
Key	Linear	MLP-2	Linear	MLP-2
`w`	0.667	0.826	0.736	0.873
`d`	0.186	0.604	0.483	0.704
`a`	0.168	0.599	0.441	0.693
`none`	0.477	0.799	0.752	0.829
`LeftArrow`	0.223	0.633	0.567	0.704
`RightArrow`	0.344	0.663	0.572	0.717
`s`	0.165	0.485	0.371	0.602
`LeftShift`	0.000	0.158	0.002	0.249
`UpArrow`	0.029	0.759	0.415	0.837
`f`	0.000	0.017	0.000	0.064

Table 16: PCA rank and Fisher along the token prefix. No prefix shows the low-rank collapse the two 32-d LAMs show in Table 19: even 8 tokens need 27 of 48 components for 80% of variance. The key result is the Fisher column: the best single dimension is coordinate 2 with score 0.132 at *every* prefix up to 128 tokens, and only the full latent finds a better one (dimension 1370). The $0.13 \rightarrow 0.40$ jump between the two VideoFlexTok Fisher entries therefore comes from one coordinate in the last eighth of the token sequence, which the 32×6 truncation discards entirely.

			components for			Fisher	
tokens	dims	PC1 (%)	80%	95%	99%	dim	score
8	48	6.35	27	40	46	2	0.132
16	96	4.03	51	77	90	2	0.132
32	192	2.70	97	153	180	2	0.132
64	384	1.91	180	297	358	2	0.132
128	768	1.63	315	570	705	2	0.132
256	1536	1.54	457	913	1218	1370	0.401

The 80%-of-variance fraction falls with width (56%, 53%, 51%, 47%, 41%, 30% of available components), so longer prefixes are progressively more redundant, but never low-rank.

Table 17: Entanglement along the token prefix. Both raw distances grow roughly six-fold across the ladder, as expected from adding dimensions, but they grow at different rates: the ratio falls monotonically from 1.72 to 1.38. The 1.38 of Table 20 is thus the *least* game-dominated point on the axis, and the 32×6 truncation is measurably worse (1.48): truncating VideoFlexTok does not merely lose action accuracy, it makes the latent more game-entangled.

tokens	dims	same A, diff G	diff A, same G	ratio
8	48	5.28	3.07	1.72
16	96	7.87	4.97	1.58
32	192	11.58	7.84	1.48
64	384	16.64	11.89	1.40
128	768	23.68	17.15	1.38
256	1536	30.66	22.26	1.38

Table 18: Per-game probing along the token prefix, mean over 193 games with the min–max range. Depth and width are substitutes here: at 8 tokens a 3-hidden-layer probe buys $+8$ points over linear, but at 256 tokens only $+0.1$. The pooled numbers of Table 15 hide how uneven this is: the worst game never rises above 45% at any width or depth, so the ladder's gains are concentrated in games that were already separable.

		linear		MLP-1		MLP-3	
tokens	dims	mean	range	mean	range	mean	range
8	48	0.63	0.30–0.95	0.70	0.38–1.00	0.71	0.37–1.00
16	96	0.68	0.32–0.97	0.75	0.37–1.00	0.75	0.35–1.00
32	192	0.73	0.36–1.00	0.78	0.42–1.00	0.78	0.40–1.00
64	384	0.78	0.40–1.00	0.81	0.42–1.00	0.81	0.41–1.00
128	768	0.82	0.41–1.00	0.83	0.43–1.00	0.83	0.42–1.00
256	1536	0.84	0.45–1.00	0.84	0.42–1.00	0.84	0.41–1.00

D Latent geometry, projections, and visual content

This section collects the analyses that characterize the latent spaces: their principal-component structure, their Fisher and entanglement geometry, their t-SNE and UMAP projections, and how much visual content decodes from them. None of these analyses directly answers a level of Q1–Q3, but they support our findings and our interpretation of the probing results.

D.1 Principal component structure

Section 3 reads the Retro low-rank result off the component counts; Table 19 and Figure 8 give the underlying curves and extend them to P2P. The collapse is a property of the encoder rather than of the corpus: four components exhaust AdaWorld on *both* corpora, and Olaf-World is less degenerate but still low-rank on both. VideoFlexTok shows no such collapse: 30% of its 1536 (256×6) components are needed for 80% of variance, and the 32×6 truncation preserves that high-rank structure at $\sim$51% (Table 16). PCA projections colored by action show action classes occupying overlapping regions of one subspace rather than separable clusters.

Table 19: Cumulative explained variance (%) from PCA.

	PC1	PC1–2	PC1–3	PC1–4
AdaWorld (P2P)	67.2	82.0	92.4	99.96
Olaf-World (P2P)	38.1	56.0	69.2	78.1
AdaWorld (Retro)	62.6	86.9	99.0	99.9
Olaf-World (Retro)	36.6	60.3	71.0	81.4
VideoFlexTok (Retro, 256×6)	1.5	2.7	3.9	4.8

D.2 Fisher discriminant and the entanglement test

No encoder's best latent coordinate comes close to the threshold of 1.0 at which between-class and within-class variance would be comparable (Table 20). The two LAMs are an order of magnitude below it on Retro, even though `right`, `left`, `crouch` and `jump` each have a direct visual consequence: what differs across games is not the action but its pixel signature, and an axis-aligned statistic pooled over 193 games cannot see a signal realized differently in each. VideoFlexTok scores several times higher and still stays under 1.

Action centroids tell the same story. On P2P they span L2 distances up to 1.07 (AdaWorld) and 1.63 (Olaf-World), the most separated pair in both being `none` against a camera movement key. On Retro the six distances between the four class means span 0.22–0.42 and 0.25–0.43 (Table 1), well inside the 0.80 and 1.07 that separate two typical samples differing only in action (row *Diff A, same G* above), so the class means differ little relative to the sample-to-sample variation. This does not by itself establish whether the classes are linearly separable. VideoFlexTok is omitted here because a centroid spread is an absolute distance and 1536-d distances are not on the scale of 32-d ones.

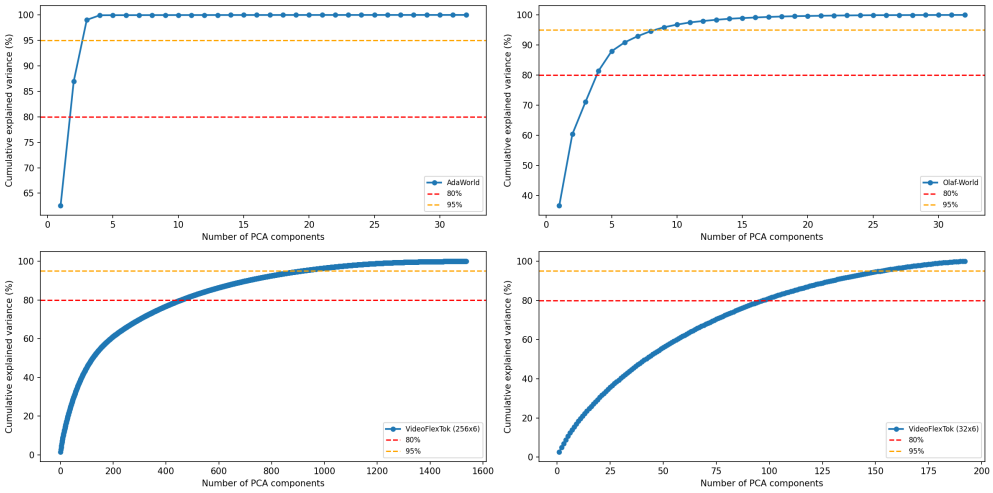


Figure 8: Cumulative PCA explained variance across latent representations: (a) AdaWorld, (b) Olaf-World, (c) VideoFlexTok, (d) VideoFlexTok (32×6).

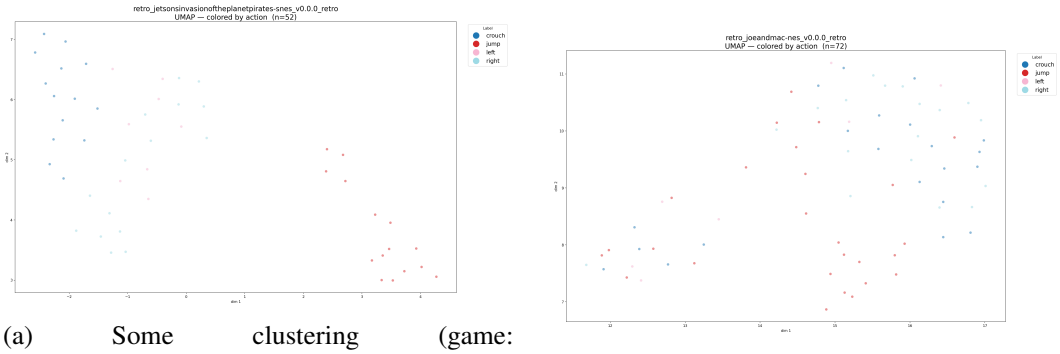
Table 20: Latent geometry: Fisher discriminant and the entanglement test. Fisher is over action classes. The entanglement block is measured on one fixed pair set per corpus, under per-action frame spacing on Retro (Section A.3); its raw distances are comparable down a column but not across encoders of different width, whereas the ratio is scale-free. Ratio < 1.0 indicates action alignment across games, > 1.0 game-identity dominance.

	P2P		Retro		
	AdaW.	OlafW.	AdaW.	OlafW.	VideoF.
Fisher	0.254	0.380	0.097	0.132	0.401
Same A, diff G	1.55	2.07	0.95	1.44	30.66
Diff A, same G	1.55	2.22	0.80	1.07	22.26
Ratio	**1.00**	**0.93**	**1.18**	**1.34**	**1.38**

D.3 Latent projections

Across both corpora the projections cluster far more strongly by game identity than by action class, most sharply for VideoFlexTok and least for AdaWorld, the same ordering as effective dimensionality above. Per-game projections show heterogeneous action structure: some games give partially separable action clusters, others substantial overlap.

Per-game UMAP



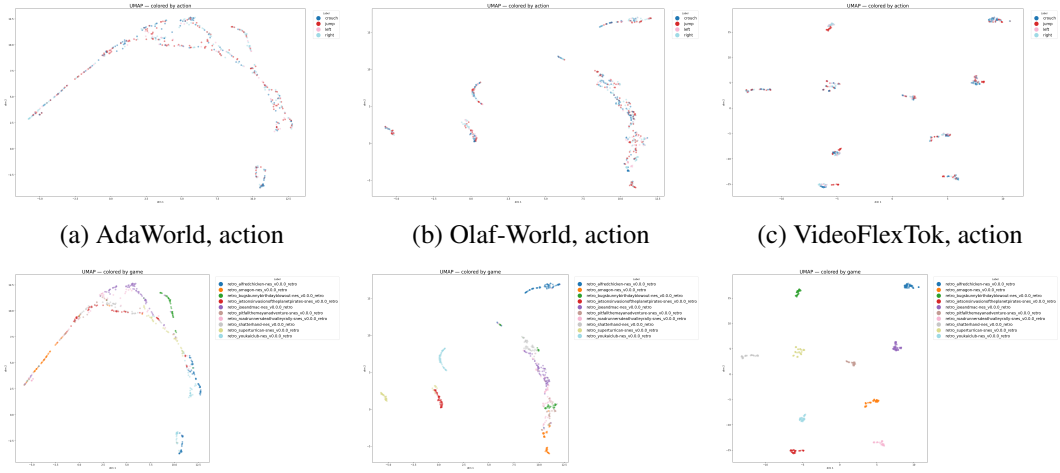
(a) Some clustering (game: jetsonsinvasionoftheplanetpirates-snes) (b) No clustering (game: joeandmac-nes)

Figure 9: Per-game UMAP visualizations of VideoFlexTok.

Figure 9 shows a UMAP visualization colored by action for two different games (82 and 83). For game 82 the actions form some clusters: jump is cleanly separated and crouch occupies its own region, with left and right interleaved in between; for game 83 no clustering is visible.

Global projections

Figure 10 projects all three encoders' Retro latents with UMAP, colored once by action and once by game. Every panel is dominated by game identity: the islands are games, and no action color holds a region of its own across any of the three. This again matches the effective-dimensionality ordering above: VideoFlexTok's game clusters are the most compact and AdaWorld's the least. t-SNE on the same latents, the same projections on P2P, and the 32×6 VideoFlexTok truncation all reproduce this and are not shown; what the projections cannot show is the within-game structure, which is what Figure 3 and the per-game fits of Section C.1 measure.



(a) AdaWorld, action (b) Olaf-World, action (c) VideoFlexTok, action

(d) AdaWorld, game (e) Olaf-World, game (f) VideoFlexTok, game

Figure 10: **Retro latents cluster by game, not by action.** One UMAP embedding per encoder, colored twice: by action class (top) and by game identity (bottom). The same structure appears under t-SNE and on P2P.

D.4 Visual content prediction

Table 21 decodes color histograms, low-resolution reconstructions and optical flow from the latents, scored as the ratio of a random-input decoder's MSE to the latent decoder's. Every cell is above 1, so

all three encoders retain substantial visual information about the frame; the ratios are several times larger on Retro than on P2P.

Table 21: Visual content prediction from latent vectors.

	P2P		Retro		
	AdaW.	OlafW.	AdaW.	OlafW.	VideoF.
Color	2.2×	2.2×	10.3×	11.7×	9.6×
Reconst.	2.6×	2.8×	8.2×	10.5×	10.1×
Optical flow	1.7×	1.9×	2.8×	3.1×	5.1×